



Step 1: 데이터 준비

목표

- CSV 데이터를 순수 **Python + NumPy**로 읽기
- **Title** , **Genre** , **Rate** 컬럼만 선택
- **Released_Year** 는 **Title** 에서 슬라이싱 추출
- 결측값 제거 후 NumPy 배열로 변환
- 데이터 크기와 데이터 형태 확인

접근 방법

1. 안전 CSV 파싱 함수 정의

- 큰 따옴표 안에 있는 쉼표는 문자열로 유지
- 쉼표 기준으로 컬럼 분리

2. 헤더에서 필요한 컬럼 인덱스 추출

- **Title** , **Genre** , **Rate** 컬럼 위치 확인

3. 데이터 줄 반복

- 안전 파싱 후 필요한 컬럼 추출
- Title 앞 숫자 제거 (**1. The Shawshank Redemption** → **The Shawshank Redemption**)
- 결측값 제거
- Title에서 Released_Year 추출
- 데이터 타입 변환 (**Rate: float** , **Released_Year: int**)

4. NumPy 배열 변환 및 확인

5. NumPy 배열 바이너리 파일로 저장

- **.npy** 파일로 저장 → 이후 **np.load** 로 쉽게 불러올 수 있음

구현 코드

```
import numpy as np

# csv 파일 경로
csv_path = "../01_data/IMDB top 1000.csv"

# csv 파싱 함수 정의
# 큰 따옴표 안에 있는 쉼표와 바깥에 있는 쉼표 구분
def parse_csv_line(line):
    row = []      # 한 줄을 분리한 문자열 담을 리스트
    current = ""   # 현재 문자열 누적
    inside_quotes = False # 큰 따옴표 안에 있는지 여부 체크
    for char in line:
        if char == '"' and not inside_quotes: # 큰 따옴표 시작
            inside_quotes = True
        elif char == '"' and inside_quotes: # 큰 따옴표 종료
            inside_quotes = False
        elif char == ',' and not inside_quotes: # 따옴표 밖에 쉼표
            row.append(current) # 여태 누적된 문자열 리스트 안에 담기
            current = ""        # 누적 초기화
        else:
            current += char      # 문자 누적
    row.append(current)         # 마지막 문자열 리스트 안에 담기
    return row                 # for문 밖에서 반환

# csv 파일 읽기
with open(csv_path,"r", encoding="utf-8") as f:
    lines = f.read().splitlines() # 줄 단위 리스트 생성

# 헤더와 데이터 분리 후
header = parse_csv_line(lines[0]) # 파싱 헤더 추출
data_lines = lines[1:]           # 데이터 줄만 따로 저장
print(header)
```

```

# 영화 제목, 장르, 평점 컬럼 인덱스 반환
title_idx = header.index("Title") # 1
genre_idx = header.index("Genre") # 4
rate_idx = header.index("Rate") # 5

# 데이터 전처리 리스트 준비
processed_data = [] # 최종 NumPy 배열로 변환할 데이터 담을 리스트

# 데이터 줄 반복 처리
for line in data_lines:
    row = parse_csv_line(line) # 파싱하여 컬럼 분리

    # row 길이가 필요한 컬럼 중 가장 큰 인덱스보다 작으면 해당 컬럼이 없다는 의미
    if len(row) <= max(title_idx, genre_idx, rate_idx):
        continue # 없다면, 건너 뛴다. 다음 줄을 처리하러 for문 처음으로 돌아감.

    # 컬럼 값 가져오기 및 공백 제거
    Num_title = row[title_idx].strip()
    genre = row[genre_idx].strip()
    rate = row[rate_idx].strip()

    # Title 앞에 있는 숫자 제거
    title = Num_title.split(':',1)[1].strip() if ':' in Num_title else Num_title

    # 결측값 제거 : 빈 문자열이 있으면 해당 행 제외
    if title == "" or genre == "" or rate == "":
        continue

    # Title에서 Released_Year 추출
    # Title 마지막에 () 형태가 있어야만 슬라이싱
    released_year = None
    if '(' in title and title[-1] == ')':
        released_year = title[title.rfind('(')+1 : title.rfind(')')]
    else:
        # 괄호가 없으면 임의로 None 처리
        released_year = None

    # Rate와 Released_Year 타입 변환 후 리스트에 추가
    try:
        processed_data.append([title, genre, float(rate), int(released_year)])
    except ValueError:
        # 변환 불가능한 값이 있으면 건너뛰기
        continue

# NumPy 배열로 변환
# dtype=object 사용 : 문자열 + 숫자가 혼합된 배열
data_array = np.array(processed_data, dtype=object)

# 결과 확인
print("NumPy 배열 형태의 데이터 출력: ", data_array)
print("\n데이터 크기(shape) 확인: ", data_array.shape)

# NumPy 배열 바이너리 파일로 저장
np.save("../04_outputs/data_array.npy", data_array)

```

1. `parse_csv_line()` 함수

- 큰 따옴표 안의 쉼표는 **데이터로 유지**
- CSV가 깨지거나 쉼표가 섞여 있어도 안전하게 분리 가능

 [parse_csv_line 함수 코드 설명](#)

2. `if len(row) <= max(title_idx, genre_idx, rate_idx): continue`

- 필요한 컬럼이 없으면 건너뛴
- 예시: `title_idx=1, genre_idx=4, rate_idx=5` → `max=5`
 - `row` 길이가 5 이하이면, Rate 컬럼이 없으므로 skip

1. Released_Year 슬라이싱

- `rfind('(')` 와 `rfind(')')` 이용 → 마지막 괄호 안 숫자 추출

- "Inception (2010)" → 2010

 [rfind\(\)는 문자열 메서드](#)

1. 결측값 처리

- 빈 문자열 또는 변환 실패 (ValueError) 시 skip

 [try를 사용하는 이유](#)

5. NumPy 배열로 변환

- NumPy에서 배열을 만들면 모든 원소가 같은 자료형이어야 효율적으로 저장하고 계산할 수 있다

 [dtype=object 사용 : 문자열 + 숫자가 혼합된 배열](#)

6. NumPy 배열(data_array)을 바이너리 파일로 저장

- 파일 확장자는 .npy 가 일반적
- 저장된 파일은 NumPy 전용 포맷이라 나중에 np.load 로 쉽게 불러올 수 있음

 [np.save 함수](#)

결과

NumPy 배열 형태의 데이터 출력: [['The Shawshank Redemption (1994)' 'Drama' 9.3 1994]
['The Godfather (1972)' 'Crime, Drama' 9.2 1972]
['The Dark Knight (2008)' 'Action, Crime, Drama' 9.0 2008]
...
['JFK (1991)' 'Drama, History, Thriller' 8.0 1991]
['Nights of Cabiria (1957)' 'Drama' 8.1 1957]
['Throne of Blood (1957)' 'Drama, History' 8.1 1957]]

데이터 크기(shape) 확인: (1000, 4)

```
NumPy 배열 형태의 데이터 출력: [['The Shawshank Redemption (1994)' 'Drama' 9.3 1994]  
['The Godfather (1972)' 'Crime, Drama' 9.2 1972]  
['The Dark Knight (2008)' 'Action, Crime, Drama' 9.0 2008]  
...  
['JFK (1991)' 'Drama, History, Thriller' 8.0 1991]  
['Nights of Cabiria (1957)' 'Drama' 8.1 1957]  
['Throne of Blood (1957)' 'Drama, History' 8.1 1957]]  
  
데이터 크기(shape) 확인: (1000, 4)  
  
종료 코드 0(으)로 완료된 프로세스
```